

CLAUDE.md

The Most Important File in Your Codebase

February 2026 · Ravn AI Development Office Hours

The Problem: Groundhog Day

Every session, you start from zero.

Without CLAUDE.md

- Re-explain the stack on every prompt
- Inconsistent code style between sessions
- Claude makes the same mistakes repeatedly
- Copy-pasting context into every conversation
- New teammates get no AI context

With CLAUDE.md

- Claude knows your project on session one
- Consistent patterns enforced automatically
- Rules persist — mistakes don't repeat
- Context loads silently, every time
- Onboarding AI context ships with the repo

What is CLAUDE.md?

A Markdown file that Claude reads **automatically** at the start of every session — no prompt required.

Persistent Context

Everything Claude needs to know about your project — written once, applied forever.

Behavioral Rules

Guard rails and conventions that survive across sessions and across teammates.

Team Knowledge

Encode what your team has learned. Every engineer gets the same AI baseline.

Platform note: CLAUDE.md works in Claude Code (CLI). It does not apply to claude.ai chat sessions.

Where Does CLAUDE.md Live?

Four scopes — each overrides the broader one below it.

Global

```
~/ .claude/CLAUDE.md
```

Project

```
./CLAUDE.md
```

Subdirectory

```
./src/components/CLAUDE.md
```

Local Override

```
./CLAUDE.local.md
```

+ Managed Policy: `/Library/Application Support/ClaudeCode/CLAUDE.md` — org-wide rules deployed by IT/DevOps, applies to all users.

Ravn AI Development Office Hours

Where Does CLAUDE.md Live?

Global

```
~/ .claude/CLAUDE.md
```

Applies to **all your projects** — every repo you open, every session you start.

Use it for personal tool preferences, universal coding habits, and default behaviors you want everywhere. Things like "always use bun" or "prefer functional style" if those apply across all your projects.

Where Does CLAUDE.md Live?

Project

```
./CLAUDE.md
```

Applies to **this codebase**. Stack conventions, architecture rules, team-wide norms.

This is the most important scope. It lives at the root of your repo and ships with your code. **Commit this**. Every teammate and every Claude session gets the same baseline.

Also supports `.claude/rules/*.md` for modular, topic-specific rules (covered below).

Where Does CLAUDE.md Live?

Subdirectory

```
./src/components/CLAUDE.md
```

Applies **only when working in** `src/components` . Module-specific patterns, domain rules, local conventions.

Use this when a specific area of the codebase has its own rules — design system patterns, component API conventions, or domain-specific logic that doesn't belong in the root CLAUDE.md.

Where Does CLAUDE.md Live?

Local Override

```
./CLAUDE.local.md
```

Gitignored. Your personal preferences that shouldn't be imposed on the rest of the team.

Use it for editor preferences, personal workflow shortcuts, or debugging habits that are yours alone. Add `CLAUDE.local.md` to your `.gitignore` — this file stays on your machine only.

The Golden Rule

"Would Claude make a mistake without this line?"

If no → delete it. If yes → keep it.

< 60

lines

Ideal

Every line earns its place. High signal, zero noise.

60-200

lines

Acceptable

Still manageable. Audit regularly for dead rules, it should be a living document.

300+

lines

Danger Zone

Context window filler. Important rules are mixed with noise and might be lost.

Why Shorter Is Better

Context Pollution

Every token in CLAUDE.md is a token **not** available for your actual work.

Long files fill Claude's context window before you've written a single prompt.

Why Shorter Is Better

Signal Dilution

When everything is a rule, nothing is.

Claude (or other AI agents) can't distinguish between important and noise. Pack as much valuable information as possible, in a concise way.

Why Shorter Is Better

Staleness

Long files drift. Having outdated information is worse than having no information at all.

The best CLAUDE.md you can write is the one you barely needed to write at all.

Less Is More

500+ line CLAUDE.md **BAD**

- Explaining what React hooks are (it has it in its training data and it's capable of reading documentation)
- "Write clean, readable code" (what does this mean? according to who?)
- "Be helpful and concise" (what would it do otherwise? prank you?)
- Outdated architecture diagrams in prose (boring!)
- Duplicating what the codebase itself shows (the best code snippets are the ones that already exist in your codebase)
- Generic best practices Claude already follows*
- * sometimes it doesn't

~40 line CLAUDE.md **GOOD**

- Always use `bun`, never `npm` or `yarn`
- Non-obvious naming: `snake_case` for DB, `camelCase` for API
- Never use `any` in TypeScript
- Run tests with: `bun test`
- What this project *does* in two sentences
- Team-specific tools and their exact invocations

Good Rules: Dos and Don'ts

✓ Specific and Actionable

Always use **bun**, never npm or yarn

Never suppress lint errors — rewrite or **ask**

New dependencies require **explicit approval**

Return **{ data, error }** from all API routes

✗ Vague or Obvious

Write clean, maintainable code **Claude's default**

Use good error handling **Too vague**

Follow TypeScript best practices **Meaningless**

Be mindful of performance **No action defined**

Rules from Our CLAUDE.md

Behavioral Constraints

Don't "improve" adjacent code, comments, or formatting.

State your assumptions explicitly. If uncertain, ask.

No "flexibility" or "configurability" that wasn't requested.

Project & Safety

Prefer JS-only libraries (no native modules) to maintain **Expo Go** compatibility.

Lint rules are immutable — never suppress (`eslint-disable` , `@ts-ignore`). Rewrite to satisfy. If impossible, ask.

Project

Task management SaaS. Next.js frontend,
Express API, PostgreSQL.

Commands

- Dev: pnpm dev
- Test: pnpm test --watch
- Lint: pnpm lint --fix

Stack & Conventions

- Use pnpm — never npm or yarn
- TypeScript strict — no `any` ever
- snake_case for DB columns, camelCase for JS
- Return { data, error } from all API routes

Do Not

- Never use console.log (use src/lib/logger)
- Never commit with --no-verify
- Never edit migrations — create new ones

1. Product context

2 sentences max. What it is, what it does.

2. Exact commands

Specific invocations, not "run tests."

3. Non-obvious conventions

Things Claude can't infer from the code.

4. Hard constraints

The "never do this" rules that protect your system.

Modular Rules with `.claude/rules/`

When CLAUDE.md starts getting long, split it — don't pad it.

Directory structure

```
.claude/  
  rules/  
    testing.md           # TDD rules, coverage requirements  
    api-conventions.md  # Return shapes, error formats  
    security.md         # No suppressions, auth patterns
```

Each file is loaded automatically — same as CLAUDE.md. One topic per file, short and auditable.

Path-specific rules via YAML frontmatter:

```
---  
paths:  
  - "src/api/**/*.ts"  
---  
  
Always return { data, error } from API routes.  
Never throw — use the error field instead.
```

Applies only when Claude works in `src/api/`. Rules for other paths are ignored.

Your root CLAUDE.md becomes an index. Domain rules live in focused files. Think of 60 lines as a soft target per file, not a hard cap.

CLAUDE.md Imports

Reference existing docs — no duplication required.

```
# Project
```

```
@README           ← pulls in your existing README
```

```
@docs/architecture.md ← team architecture doc
```

```
@.claude/rules/testing.md ← a specific rules file
```

How it works

- Paths are relative to the importing file
- Recursive imports (up to 5 levels deep)
- Works in CLAUDE.md and rules files alike

Why it matters

If your team already wrote `docs/git-workflow.md`, import it. Don't rewrite it in CLAUDE.md. Single source of truth.

The Self-Improving Pattern

Your CLAUDE.md should evolve as you work — not stay frozen.

1 Work normally with Claude Code

Claude hits an edge case or makes an assumption you need to correct.

2 Notice the pattern

Did you rephrase the same correction twice? That's a CLAUDE.md candidate.

3 Ask Claude to remember it: `Add this to CLAUDE.md: Always follow red green TDD before implementing a new component`

4 Audit the rule regularly

Is it still true? Can it be rephrased? Remove stale rules aggressively.

5 Commit it to the repo

CLAUDE.md is team infrastructure. It ships with the code.

Common Mistakes

Explaining things Claude knows

"React is a UI library for building component-based interfaces..."

Fix: Content like this is already part of Claude's training data. Only add what's specific to *your* project.

Rules that contradict each other

"Always use functional components" + "Use class components for error boundaries"

Fix: Be explicit about exceptions. Ambiguity leads to inconsistency.

No ownership = no updates

A 300-line CLAUDE.md created once but never updated.

Fix: Establish a routine of maintaining it, suggest improvements in code reviews, share suggestions with your team

Mixing global and project rules

Personal preferences in the project CLAUDE.md imposed on the whole team.

Fix: Personal preferences (project) → `CLAUDE.local.md` . Team rules → `CLAUDE.md` .

Getting Started Is Easy

Three steps

1 Run `/init` in your project

Claude scans the repo and auto-generates a starting CLAUDE.md.

2 Trim the fat

Remove anything obvious. Add your real conventions. Target under 100 lines and try to get more concise over time.

3 Commit it

It's team infrastructure. Ship it with the code.

What `/init` produces

```
$ claude
> /init

Scanning project structure...
Detected: TypeScript, React Native, bun
Found: package.json, tsconfig.json

✓ Created CLAUDE.md (34 lines)

Review and customize before committing.
```

CLAUDE.md vs Skills vs Rules

	CLAUDE.md	Rules	Skills
Purpose	Persistent project context	Topic-specific rule files	Reusable prompt workflows
When active	Every session, automatically	Every session, automatically	When explicitly invoked
Written in	Markdown	Markdown + optional YAML frontmatter	Markdown (prompt templates)
Lives in	<code>./CLAUDE.md</code> , <code>~/.claude/CLAUDE.md</code>	<code>.claude/rules/*.md</code>	<code>~/.claude/</code>
Best for	Stack, conventions, commands	Keeping CLAUDE.md concise	Repeated multi-step tasks
Committed?	Yes — team infrastructure	Yes	Optional

All three compose. CLAUDE.md is the foundation everything builds on.

10 Best Practices

- 1 Aim for around 60 lines**
Every line is more token usage. It's a guideline, not a hard cap — but make them count.
- 2 Only write what Claude can't infer**
Trust training. Add project-specifics only.
- 3 Use specific, actionable language**
"Use bun" not "use the right package manager."
- 4 Include exact commands**
Not "run tests" — the actual invocation.
- 5 Version control it**
It ships with the repo. Always.
- 6 Ask Claude to add rules as you work**
e.g. "Add this to CLAUDE.md: always use bun, never npm."
- 7 Audit periodically**
Delete stale rules ruthlessly.
- 8 Layer your scopes**
Global for personal, project for team.
- 9 Use `CLAUDE.local.md` for personal rules**
Don't impose your preferences on the team.
- 10 Treat it as living documentation**
If the codebase changed, update the file.

Your Action Plan

Today Run `/init` in your main project

Let Claude generate a starting point. Takes 30 seconds.

Day 2-3 Trim and customize

Delete what's obvious. Add your real conventions. Aim for around 60 lines — treat it as a signal, not a ceiling.

Week 1 Commit and share with your team

Open a PR. Invite teammates to review and add their conventions.

Ongoing Evolve it

Every time you correct Claude twice, it's a CLAUDE.md candidate.

Quarterly Audit and prune

Delete stale rules. The goal is fewer lines, not more.

Two Types of Memory

CLAUDE.md is what **you** write for Claude. Auto memory is what **Claude** writes for itself.

CLAUDE.md (you write)

- Project context, team rules, commands
- Committed to the repo
- Applies to all teammates
- You control every line

Auto Memory (Claude writes)

- Learnings Claude picks up from your sessions
- Lives in `~/.claude/projects/<project>/memory/`
- Private — never committed to the repo
- Claude updates it automatically as you work

Auto memory is Claude's notes to itself. CLAUDE.md is the team rulebook you set intentionally. You don't need to manage auto memory — just know it exists.

Start with one file.

The best CLAUDE.md you can write today is a short one — 20 lines that capture exactly what Claude would get wrong without them.

Run `/init` today

Target ~60 lines

Ship it with your code

Questions? Ask in **#ai** on Slack · docs.anthropic.com/claude/docs/claude-md

Ravn AI Development Office Hours